

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки/специальность
09.03.01 Информатика и вычислительная техника

направленность (профиль)/специализация
«Технологии разработки программного обеспечения»

Выпускная квалификационная работа

Разработка инструмента для восстановления
базы данных MySQL на точку во времени

Обучающегося 4 курса
очной формы обучения
Фирсова Кирилла Александровича

Руководитель выпускной квалификационной
работы:
Кандидат педагогических наук, доцент
Государев Илья Борисович

Рецензент:

Ученая степень (при наличии), ученое звание
(при наличии), должность
Ф. И. О. (указывается в именительном
падеже)

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ	4
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ СРЕДСТВ РЕЗЕРВНОГО КОПИРОВАНИЯ.....	7
1.1. Резервное копирование и восстановление баз данных как элемент отказоустойчивости.....	7
1.2. Архитектура MySQL, транзакционность и журналирование изменений.....	9
1.3. Восстановление на точку во времени и роль бинарных журналов	11
1.4. Сравнительный анализ средств резервного копирования MySQL.	13
1.5. Постановка проблемы и выводы по главе 1	16
ГЛАВА 2. ПРОЕКТИРОВАНИЕ, РАЗРАБОТКА И ТЕСТИРОВАНИЕ ИНСТРУМЕНТА ВОССТАНОВЛЕНИЯ БАЗ ДАННЫХ.....	19
2.1. Требования к инструменту восстановления	19
2.2. Архитектура решения на основе WAL-G и компонента binlog-server	22
2.3. Алгоритм восстановления базы данных на заданный момент времени.....	24
2.4. Реализация компонентов загрузки, хранения и отдачи бинарных журналов.....	27
2.5. Порядок запуска и предварительная настройка wal-g binlog-server	29
2.6. Методика тестирования и анализ результатов	30
2.7. Оценка ограничений, рисков и направлений развития.....	34
ЗАКЛЮЧЕНИЕ.....	36
СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ.....	38
СЛОВАРЬ ТЕРМИНОВ.....	39
СПИСОК ЛИТЕРАТУРЫ	40
СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА	42

ПРИЛОЖЕНИЯ	43
------------------	----

ВВЕДЕНИЕ

Актуальность темы разработки инструмента для восстановления базы данных MySQL на заданный момент времени обусловлена возрастающими требованиями к надежности, доступности и сохранности данных в современных информационных системах. В условиях постоянного увеличения объёмов хранимой и обрабатываемой информации потеря даже небольшой части данных вследствие сбоев оборудования, ошибок программного обеспечения или некорректных действий пользователей может привести к значительным финансовым и организационным последствиям. В связи с этим особое значение приобретают технологии, обеспечивающие быстрое и точное восстановление данных.

Одной из наиболее распространённых систем управления базами данных является MySQL, широко применяемая как в корпоративных информационных системах, так и в облачных сервисах. Несмотря на наличие встроенных механизмов резервного копирования и бинарного журналирования транзакций, стандартные средства MySQL не обеспечивают в полной мере автоматизированного и удобного восстановления базы данных на произвольный момент времени (Point-in-Time Recovery), что особенно критично для высоконагруженных сервисов с непрерывным потоком транзакций.

Необходимость разработки специализированного инструмента подтверждается его практической востребованностью в промышленной эксплуатации. Заказчиком данной разработки выступает Яндекс, использующий подобные механизмы в собственном облачном сервисе DBaaS MySQL. Для сервисов класса Database as a Service обеспечение гарантированного восстановления клиентских баз данных на заданный момент времени является одной из ключевых функций, напрямую влияющих на отказоустойчивость и качество предоставляемых услуг. Следовательно,

разработка инструмента носит не только исследовательский, но и прикладной характер, ориентированный на решение реальной производственной задачи.

Дополнительную актуальность теме придаёт необходимость эффективной работы с бинарными логами: их непрерывной загрузки, централизованного хранения, передачи в удалённые хранилища и последующего воспроизведения при восстановлении. Использование современных средств резервного копирования, таких как WAL-G, создаёт технологическую основу для решения данной задачи, однако требует расширения функциональности путём разработки специализированного компонента binlog-server, обеспечивающего обработку и подготовку бинарных журналов для сценариев Point-in-Time Recovery.

Таким образом, актуальность выбранной темы определяется сочетанием нескольких факторов: высокой значимостью обеспечения сохранности данных, ограниченностью стандартных средств MySQL, востребованностью подобных решений в облачной инфраструктуре и наличием конкретного индустриального заказчика в лице Яндекса, что подтверждает практическую ценность и перспективность проводимой разработки.

Цель: Разработка и реализация инструментария для восстановления базы данных MySQL на заданный момент времени с использованием бинарных логов и интеграции с существующими решениями резервного копирования, направленного на повышение отказоустойчивости и надежности систем хранения данных.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ теоретических основ резервного копирования и восстановления баз данных, включая архитектуру СУБД, транзакционные модели, журналирование операций и механизмы обеспечения отказоустойчивости;

2. Исследовать и сравнить существующие инструменты резервного копирования и механизмы восстановления, в частности, использование бинарных логов в MySQL, выявив их преимущества и недостатки;
3. Спроектировать архитектуру инструмента для эффективного хранения бинарных логов и восстановления баз данных, определив требования, ограничения, сценарии использования и модель хранения данных;
4. Разработать алгоритм восстановления базы данных на заданную точку времени, включающий последовательность действий, обработку временных меток и воспроизведение транзакций из бинарных логов;
5. Реализовать спроектированный инструмент, включая модули для загрузки, хранения и применения бинарных логов, а также обеспечения отказоустойчивости и обработки ошибок;
6. Разработать интерфейс взаимодействия (cli интерфейс) с инструментом, позволяющий выполнять основные операции с заданными параметрами;
7. Провести тестирование разработанного решения в среде, приближенной к реальной, оценив его эффективность по критериям времени восстановления, нагрузке на систему и целостности данных при различных сценариях сбоев;
8. Сравнить результаты тестирования разработанного инструмента с существующими решениями для подтверждения его практической значимости и эффективности;

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ СРЕДСТВ РЕЗЕРВНОГО КОПИРОВАНИЯ

В первой главе рассматриваются механизмы, на которых строится восстановление MySQL на заданный момент времени: резервные копии, журналы изменений, бинарные журналы, GTID и инструменты восстановления. Для данной работы важно не просто перечислить существующие средства, а показать, где именно появляется узкое место при PITR: в создании полной копии, в доставке бинарных журналов или в их последующем применении.

Практика показывает, что наличие backup еще не означает готовность к восстановлению. У администратора должен быть полный набор данных: сама копия, точка старта в binary log, непрерывная цепочка журналов до целевой точки, сведения о GTID и понятный сценарий проверки результата. Если один из этих элементов отсутствует, PITR превращается в ручной разбор аварии.

1.1. Резервное копирование и восстановление баз данных как элемент отказоустойчивости

Резервное копирование базы данных - это не только сохранение файлов. Для MySQL резервная копия должна быть согласованной: после разворачивания сервер должен открыть datadir, восстановить внутренние журналы InnoDB и продолжить работу без нарушения транзакционных границ. Кроме того, для восстановления на момент позже полной копии должны быть доступны бинарные журналы.

В эксплуатации обычно используют два показателя: RPO и RTO. RPO показывает, сколько данных допустимо потерять. RTO показывает, сколько времени система может оставаться недоступной. В контексте PITR второй показатель часто оказывается более болезненным. Полная копия может

развернуться достаточно быстро, но затем сервер еще долго применяет binlog. Если это занимает часы, формально восстановление возможно, но для пользователя сервис все равно недоступен слишком долго.

Для MySQL применяются разные типы резервного копирования. Логическое копирование формирует SQL-дамп структуры и данных. Физическое копирование переносит файлы таблиц и служебные файлы движка хранения. Инкрементальное копирование сохраняет изменения относительно базовой копии. Непрерывное архивирование бинарных журналов позволяет довести восстановленное состояние до момента после создания полной копии [4; 8].

Логический backup удобен для переноса отдельных таблиц и миграции между версиями, но на больших объемах данных он часто проигрывает по скорости. Физический backup обычно быстрее для крупных баз, однако требует корректной подготовки копии и совместимости с используемым движком хранения. Для PITR решающим становится не только способ создания полной копии, но и скорость применения журналов после нее [8; 9].

Отдельно нужно учитывать временное дисковое пространство. Во время восстановления на одном узле могут одновременно находиться восстановленная директория данных, служебные файлы MySQL, скачанные бинарные журналы, временные файлы распаковки и диагностические логи. Если диапазон восстановления включает крупные binlog-файлы, например более 15 ГБ каждый, предварительное скачивание всей цепочки может потребовать десятки или сотни гигабайт сверх размера самой базы.

Надежная процедура восстановления должна включать проверку резервной копии, запуск MySQL, контроль схемы, выборочные проверки критичных таблиц, анализ ошибок и подтверждение доступности нужного диапазона binary log. Проверка только факта успешного скачивания backup недостаточна: восстановление может остановиться позднее, на этапе применения журналов.

Ошибки восстановления полезно разделять на повторяемые и критические. Временный сетевой сбой или недоступность хранилища можно обработать повторной попыткой. Разрыв цепочки бинарных журналов, повреждение резервной копии, несовместимость версии MySQL или конфликт GTID должны останавливать процедуру до применения данных. Такое разделение важно, потому что при PITR ошибка в середине процесса может привести не к отказу запуска, а к некорректному состоянию базы.

Бинарные журналы могут содержать коммерчески значимые и персональные сведения, поскольку в них фиксируются изменения данных. Поэтому служебные логи инструмента восстановления не должны без необходимости сохранять значения строк или текст SQL-событий. Достаточно фиксировать технические атрибуты: имя binlog-файла, позицию, GTID-множество, размер, контрольную сумму и текущий этап восстановления.

1.2. Архитектура MySQL, транзакционность и журналирование изменений

MySQL использует несколько типов журналов. Redo log нужен InnoDB для восстановления после аварийного завершения. Undo log используется для отката транзакций и реализации многоверсионности. Binary log фиксирует изменения, которые могут быть переданы репликам и использованы при восстановлении на точку во времени [6]. Эти журналы нельзя заменять друг другом: redo log помогает поднять локальный экземпляр после падения, а binary log нужен для продвижения восстановленной копии к более позднему состоянию.

Транзакция в реляционной СУБД должна обладать свойствами атомарности, согласованности, изолированности и долговечности. Для восстановления это имеет прямое значение: нельзя применить только часть транзакции и считать результат корректным. Если во время PITR нарушается

граница транзакции, восстановленная база может стартовать, но данные будут логически неверными [14; 16].

Бинарный журнал MySQL представляет собой последовательность файлов с числовыми суффиксами. При включенном binary logging сервер записывает в него события, изменяющие данные. Документация MySQL указывает, что binary log используется для резервного копирования и репликации, а файлы по умолчанию создаются в каталоге данных, если иной путь не задан параметром --log-bin [6].

Формат записи событий может быть statement-based, row-based или mixed. В MySQL 8.4 формат row-based является значением по умолчанию, а поддержка других форматов рассматривается как устаревающая для новых репликационных конфигураций [6]. Для восстановления row-based формат обычно предпочтительнее: он описывает изменения строк и снижает риск расхождения из-за недетерминированных SQL-выражений.

Отдельное значение имеют GTID - глобальные идентификаторы транзакций. GTID связывает транзакцию с источником и последовательным номером. MySQL сохраняет GTID при репликации, а повторное выполнение уже зафиксированной транзакции с тем же GTID игнорируется [7]. Для инструмента восстановления это удобно: можно контролировать, какие транзакции уже входят в полную копию, а какие еще должны быть применены.

Состояние базы при PITR можно представить как последовательность S0, S1, ..., St. Состояние S0 фиксируется полной резервной копией. Дальше состояние меняется за счет событий из бинарных журналов. Если целевая точка задана временем, выбираются события до указанной временной границы. Если она задана позицией, используются имя binlog-файла и offset. Если восстановление выполняется по GTID, граница описывается множеством транзакций.

Важно, что успешный запуск MySQL после восстановления полной копии еще не доказывает успешность PITR. Сервер может стартовать в

состоянии на момент backup, но не дойти до нужной точки, если binlog не были применены или применились не полностью. Поэтому итоговая проверка должна подтверждать не только запуск сервера, но и достижение целевого момента.

1.3. Восстановление на точку во времени и роль бинарных журналов

Point-in-Time Recovery - это восстановление состояния базы данных на конкретный момент времени. В MySQL оно обычно состоит из двух этапов: разворачивание полной резервной копии и применение бинарных журналов, созданных после нее [4]. Именно второй этап позволяет восстановить состояние не только на момент backup, но и на более позднюю точку.

Официальная документация MySQL описывает использование утилиты mysqlbinlog для чтения бинарных журналов и применения их к серверу. Утилита может выбирать диапазон событий по времени через --start-datetime и --stop-datetime либо по позициям через --start-position и --stop-position [5]. Это базовый и понятный способ восстановления. Его удобно использовать в небольших сценариях, при анализе отдельных событий или при ручном разборе инцидента.

Восстановление по времени удобно, когда известен момент ошибочной операции. Например, если некорректный DELETE был выполнен в 13:10, целевая точка может быть выбрана до этой операции. Но временная граница зависит от точности часов, временной зоны и корректного определения момента ошибки. Восстановление по позиции точнее технически, но требует предварительного анализа журнала. Восстановление по GTID удобно в средах, где все транзакции имеют глобальные идентификаторы [7].

Ограничения классического восстановления становятся заметны при больших журналах. Сначала нужно получить нужные binlog-файлы, затем прочитать их, преобразовать в применимый поток команд или событий и

передать в MySQL. При небольшом объеме это почти не влияет на итоговое время. При десятках гигабайт binlog эта часть становится самостоятельным этапом восстановления и увеличивает RTO.

WAL-G частично автоматизирует этот процесс. Он умеет загружать binlog в хранилище, скачивать их обратно и запускать восстановление через binlog-replay [10]. Это важная возможность, потому что она убирает часть ручной работы. Однако сам факт наличия binlog-replay не решает полностью проблему скорости и больших файлов. Если восстановлению нужно пройти длинный диапазон журналов или обработать отдельные binlog размером более 15 ГБ, существенными остаются накладные расходы на чтение, передачу и промежуточное представление данных.

Еще одна практическая проблема связана с локальным диском. Наиболее простой сценарий - сначала скачать все нужные binlog, затем начать применение. Он удобен для реализации, но плохо масштабируется: временный каталог должен вместить весь диапазон восстановления. Если журналы скачиваются только перед непосредственным применением и удаляются после него, требования к локальному диску становятся значительно ниже.

Percona XtraBackup описывает PITR как восстановление полной директории данных и последующее применение бинарных журналов от позиции, записанной в файле xtrabackup_binlog_info [8]. Это подтверждает общий принцип: физическая копия сама по себе недостаточна для восстановления на произвольный момент после ее создания. Нужны бинарные журналы, корректная стартовая позиция и механизм применения событий.

PITR особенно полезен при логических ошибках. Аппаратный сбой обычно требует вернуть последнюю доступную копию. Логическая ошибка сложнее: неверный UPDATE или DELETE может быть корректно выполнен MySQL, попасть в репликацию и оказаться в последующих backup. В таком

случае требуется восстановить состояние непосредственно перед ошибочной операцией, а не просто поднять последнюю копию.

Таким образом, для больших баз данных главная сложность PITR находится не только в наличии резервных данных. Не менее важны два эксплуатационных параметра: время применения бинарных журналов и максимальный размер временных файлов на узле восстановления. Эти параметры напрямую влияют на то, можно ли использовать PITR в реальном аварийном окне.

1.4. Сравнительный анализ средств резервного копирования MySQL

Сравнение средств резервного копирования MySQL нужно проводить по нескольким признакам: способ создания копии, пригодность для PITR, работа с binary log, поддержка удаленного хранилища, скорость восстановления и требования к временным файлам. В рамках данной работы особенно важен последний участок процесса - применение бинарных журналов после разворачивания полной копии.

mysqldump является штатной утилитой логического резервного копирования. Ее достоинства - простота, читаемый SQL-результат и переносимость между окружениями. Недостаток очевиден на больших базах: создание и восстановление дампа может занимать значительное время. Кроме того, mysqldump сам по себе не решает задачу непрерывного архивирования и применения binary log [4; 5].

MyDumper развивает идею логического копирования за счет параллельного экспорта и импорта. Документация проекта отмечает, что MyDumper возник как альтернатива однопоточному mysqldump и использует параллелизм для ускорения backup/restore [13]. Но это все еще инструмент логической копии, а не полноценный механизм PITR с управлением binlog-цепочкой.

Percona XtraBackup является распространенным средством физического резервного копирования MySQL. Для PITR важна его способность

записывать xtrabackup_binlog_info с именем бинарного журнала и позицией, соответствующими резервной копии [9]. XtraBackup хорошо решает задачу получения согласованной физической копии, но не заменяет отдельную логику доставки и применения последующих бинарных журналов.

WAL-G закрывает значительную часть задач вокруг хранения. Для MySQL он поддерживает работу с резервными копиями, объектными хранилищами и бинарными журналами, включая команды binlog-push, binlog-fetch и binlog-replay [10; 11]. Это делает WAL-G удобной основой для PITR. При этом узкое место остается в способе применения больших binlog: если восстановление фактически превращается в долгую последовательную обработку крупных файлов, RTO может оставаться неприемлемым. В таблице 1 приведен подробный сравнительный анализ.

Таблица 1 – Сравнительный анализ средств резервного копирования и восстановления MySQL

Средство	Тип копии	PITR	Преимущества	Ограничения
mysqldump	Логическая	Через binlog и mysqlbinlog	Простота, переносимость, читаемый SQL	Низкая скорость на больших объемах, нет управления binlog-цепочкой

Средство	Тип копии	PITR	Преимущества	Ограничения
MyDumper/MyLoader	Логическая параллельная	Через binlog и метаданные	Параллелизм, быстрее mysqldump на ряде сценариев	Не решает хранение и применение binlog
Percona XtraBackup	Физическая	Через xtrabackup_binlog_info и binlog	Быстрое физическое копирование, фиксация стартовой позиции	Требует подготовки копии и отдельного применения журналов
WAL-G binlog-replay	Потоковая/физическая + binlog	Через хранение и replay бинарных журналов	Объектное хранилище, сжатие, шифрование, автоматизация доставки	При больших binlog остаются проблемы скорости применения и

Средство	Тип копии	PITR	Преимущества	Ограничения
				временного диска

Из сравнения видно, что существующие средства закрывают разные части задачи. `mysqldump` и `MyDumper` удобны как логические инструменты. `XtraBackup` хорошо подходит для физического backup. `WAL-G` полезен для хранения и доставки резервных данных. Но для восстановления на произвольную точку при большом объеме изменений нужен связанный сценарий, который управляет цепочкой от выбора полной копии до контролируемого применения `binlog`.

При проектировании такого сценария нельзя считать, что проблема заканчивается на скачивании файлов из хранилища. Для больших баз решающими становятся скорость `roll-forward` и максимальный объем временного каталога. Поэтому инструмент восстановления должен отдельно контролировать не только наличие бинарных журналов, но и то, как именно они передаются восстановленному MySQL.

Также важно избегать опасных действий над исходной базой. Восстановление безопаснее выполнять в отдельный экземпляр, затем проверять данные и только после этого принимать решение о переключении приложения или переносе нужных таблиц. Такой подход снижает риск усугубить исходный инцидент.

1.5. Постановка проблемы и выводы по главе 1

Проведенный анализ показывает, что MySQL уже содержит базовые механизмы PITR: бинарные журналы, позиции, GTID и утилиты для чтения журналов [4; 5; 7]. `WAL-G` дополняет эту схему хранением резервных копий и бинарных журналов во внешнем хранилище, а также командами `binlog-`

fetch и binlog-replay [10]. Следовательно, проблема данной работы не состоит в отсутствии PITR как такового.

Основная проблема состоит в эксплуатационной стоимости восстановления. После разворачивания полной копии база еще должна пройти путь от состояния backup до целевой точки. Если между этими моментами накопился большой объем binary log, именно этот этап начинает определять итоговый RTO. Особенно заметно это при крупных журналах, например файлах более 15 ГБ, когда любое лишнее преобразование и предварительное скачивание полного диапазона увеличивает время и требования к диску.

Классический подход через mysqlbinlog надежен и полезен как базовый механизм, но он не оптимален для длинного потока изменений. WAL-G binlog-replay автоматизирует часть работы с журналами, однако не устраняет саму проблему применения большого объема binlog. Для практического инструмента восстановления нужно уменьшить число промежуточных преобразований, отказаться от формирования цельного SQL-представления и не хранить локально весь диапазон журналов одновременно.

Таким образом, основная задача дальнейшей разработки формулируется следующим образом: создать инструмент восстановления MySQL на заданный момент времени, который использует резервные копии и бинарные журналы из внешнего хранилища, применяет binlog поэтапно, сокращает время roll-forward и ограничивает потребление временного дискового пространства.

Выводы по главе 1: PITR для MySQL основан на восстановлении полной копии и применении бинарных журналов; ключевыми метаданными являются имя binlog, позиция, GTID-множество и целевая временная точка; WAL-G уже предоставляет механизмы хранения и replay бинарных журналов; основными нерешенными ограничениями остаются скорость применения больших binlog и расход локального диска; дальнейшая разработка должна быть направлена на передачу журналов восстановленному

MySQL без промежуточного цельного SQL-файла и без полной локальной материализации всей цепочки.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ, РАЗРАБОТКА И ТЕСТИРОВАНИЕ ИНСТРУМЕНТА ВОССТАНОВЛЕНИЯ БАЗ ДАННЫХ

Во второй главе описываются проектирование, разработка и тестирование инструмента восстановления базы данных MySQL на заданный момент времени. Инструмент рассматривается как расширение WAL-G методом binlog-server, обеспечивающим отдачу бинарных журналов восстановленному серверу MySQL по репликационному протоколу.

Проектирование велось с учетом требований промышленной эксплуатации: отказоустойчивость, наблюдаемость, воспроизводимость, проверяемость результата, безопасность учетных данных и возможность запуска в составе штатной команды wal-g binlog-server. В отличие от разовой административной инструкции, инструмент должен быть пригоден для многократного запуска, автоматического завершения, повторного старта после ошибки и регистрации всех значимых этапов восстановления.

2.1. Требования к инструменту восстановления

Требования к инструменту были сформированы на основании анализа классического PITR, возможностей WAL-G и практической проблемы больших бинарных журналов. Базовый сценарий таков: пользователь выбирает резервную копию и целевой момент времени; система создает новый экземпляр MySQL; разворачивает резервную копию; подготавливает GTID или позицию; запускает binlog-server; настраивает восстановленный MySQL как реплику; затем применяет события до целевой точки через репликационный протокол. Отдельное требование - работа с крупными binlog-файлами, в том числе размером более 15 ГБ, без предварительного перевода всего содержимого в SQL и без скачивания всей цепочки журналов на локальный диск.

Таблица 2 – Функциональные и нефункциональные требования к инструменту

Группа	Требование	Описание
Функциональные	Выбор точки восстановления	Время, позиция binlog или GTID-граница
Функциональные	Получение резервной копии	Использование WAL-G backup-fetch или внешнего restore-механизма
Функциональные	Отдача бинарных журналов	binlog-server отдает MySQL события по протоколу репликации
Функциональные	Контроль завершения	Остановка при достижении целевой точки
Нефункциональные	Идемпотентность	Повторный запуск не приводит к двойному применению GTID
Нефункциональные	Безопасность	Секреты не попадают в журналы, доступ разграничивается ролями
Нефункциональные	Наблюдаемость	Логи и метрики показывают этап восстановления
Нефункциональные	Совместимость	MySQL 8.0/8.4, row-based binlog как предпочтительный формат

Требования к корректности являются приоритетными. Инструмент не должен применять события, относящиеся к моменту после целевой точки. Если в цепочке бинарных журналов обнаружен разрыв, восстановление должно завершиться ошибкой до запуска применения, а не создавать потенциально неконсистентную базу.

Особое внимание уделяется идемпотентности. Восстановление может прерваться из-за сетевой ошибки, недоступности объектного хранилища, перезапуска контейнера или сбоя MySQL. Повторный запуск не должен начинать процесс с неопределенного состояния; для этого состояние операции сохраняется в виде набора этапов: `backup_selected`, `backup_restored`, `mysql_started`, `gtid_initialized`, `binlog_server_started`, `replication_configured`, `logs_applied`, `validation_done`.

Нефункциональные требования связаны с безопасностью и эксплуатацией. Учетные данные к MySQL, объектному хранилищу и пользователю репликации не должны попадать в открытые журналы. Поскольку восстановление может использовать значительный объем сетевого и дискового ввода-вывода, инструмент должен поддерживать настройки лимитов, параллелизма и временных каталогов.

При сравнении подходов необходимо отдельно фиксировать время восстановления полной копии и время применения бинарных журналов. Именно второй показатель показывает эффект от отказа от формирования цельного SQL-представления и от поэтапной загрузки binlog. Дополнительно измеряется максимальный объем временного каталога: для разработанного сценария он должен оставаться близким к размеру одного текущего binlog-файла или небольшого буфера, а не к суммарному объему всех журналов в диапазоне восстановления.

Для практического применения важно учитывать, что восстановление является не только технической процедурой, но и управляемым процессом эксплуатации. Пользователь или оператор должен задать резервную копию и целевую точку восстановления, а инструмент обязан выполнить проверку

исходных данных, применение журналов и валидацию результата без ручного подбора команд MySQL.

2.2. Архитектура решения на основе WAL-G и компонента binlog-server

Архитектура решения построена вокруг трех основных компонентов: Recovery Manager, WAL-G binlog-server и восстановленный экземпляр MySQL. Recovery Manager управляет сценарием восстановления: принимает параметры, проверяет доступность резервной копии и журналов, запускает команды WAL-G, формирует конфигурацию MySQL, отслеживает прогресс и сохраняет состояние операции. На рисунке 1 представлена схема использования binlog-server.

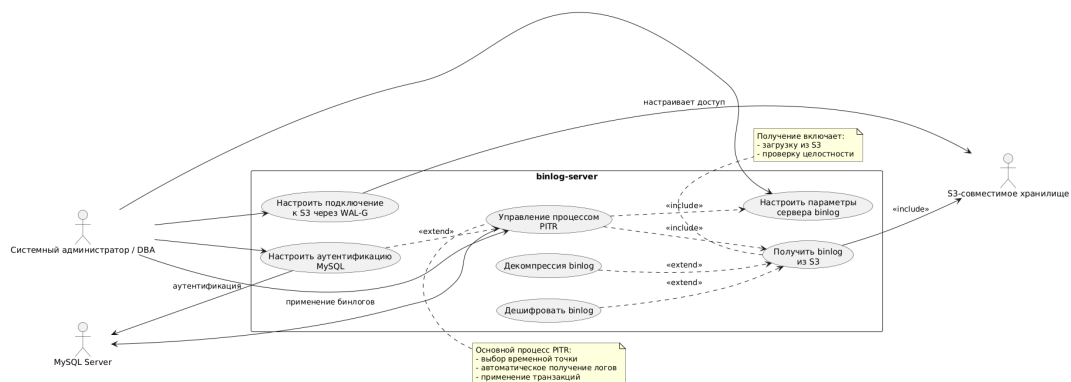


Рисунок 1 – Use Case диаграмма восстановления MySQL на точку во времени с использованием WAL-G binlog-server

S3 хранит два типа данных: полные резервные копии и бинарные журналы. Полная копия используется как исходное состояние. Бинарные журналы используются для продвижения состояния к целевой точке. WAL-G обеспечивает загрузку и выгрузку данных в хранилище, сжатие и, при необходимости, шифрование [7].

Recovery Manager реализован как управляющий слой. Его функции включают валидацию входных параметров, выбор подходящей резервной копии, проверку непрерывности binlog, генерацию временной конфигурации,

запуск процесса восстановления, обработку ошибок, запись состояния, вывод служебных сообщений и фиксацию этапов операции.

binlog-server запускается как отдельный процесс. Он слушает TCP-порт, принимает подключение от восстановленного MySQL, идентифицирует позицию или GTID, с которой нужно начать передачу, извлекает соответствующие бинарные журналы из объектного хранилища и отдает события по протоколу репликации. Согласно документации WAL-G, режим binlog-server может использоваться для получения binlog из хранилища и отправки их MySQL slave по репликационному протоколу [7].

Ключевое отличие выбранного подхода от классического сценария с mysqlbinlog и от восстановления через цельный SQL-поток состоит в том, что бинарные журналы не переводятся в большой промежуточный SQL-файл. Восстановленный MySQL подключается к binlog-server как реплика и получает события через штатный протокол репликации. Далее их применяет replication applier MySQL, то есть используется уже существующий механизм сервера, а не внешний клиент, который последовательно скапливает SQL-команды. Это уменьшает накладные расходы на преобразование событий и особенно важно на длинных диапазонах восстановления.

Второе важное преимущество связано с использованием временного дискового пространства. Инструменту не требуется заранее скачивать всю цепочку бинарных журналов от полной резервной копии до целевой точки. Очередной binlog извлекается из хранилища непосредственно перед передачей восстановленному MySQL, после применения удаляется из временного каталога, а затем загружается следующий файл. За счет этого можно работать с крупными журналами, включая файлы больше 15 ГБ: локальный диск занят текущим рабочим набором, а не всем диапазоном восстановления.

Восстановленный MySQL сначала стартует из состояния полной резервной копии. Если используется GTID, перед началом применения

журналов необходимо корректно выставить `gtid_purged`. WAL-G документация приводит пример, где значение GTID берется из `xtrabackup_binlog_info` и устанавливается после `RESET MASTER` [7]. При этом `binlog-server` не отменяет этап предварительной настройки: до запуска восстановления должны быть заданы адрес и порт сервера, `server_id`, учетные данные, источник репликации, временный каталог для `binlog` и параметры доступа к хранилищу.

2.3. Алгоритм восстановления базы данных на заданный момент времени

Алгоритм восстановления начинается с проверки параметров и конфигурации. Целевая точка должна быть задана в однозначном формате времени, предпочтительно RFC3339 или UTC-время, что снижает риск ошибки из-за временных зон. Если выбран режим восстановления по позиции, дополнительно указываются имя `binlog`-файла и позиция; если выбран режим GTID, указывается GTID-множество или граница применения. До старта также проверяются параметры `binlog-server`: порт, `server_id`, учетная запись репликации, доступ к объектному хранилищу и путь временного каталога.

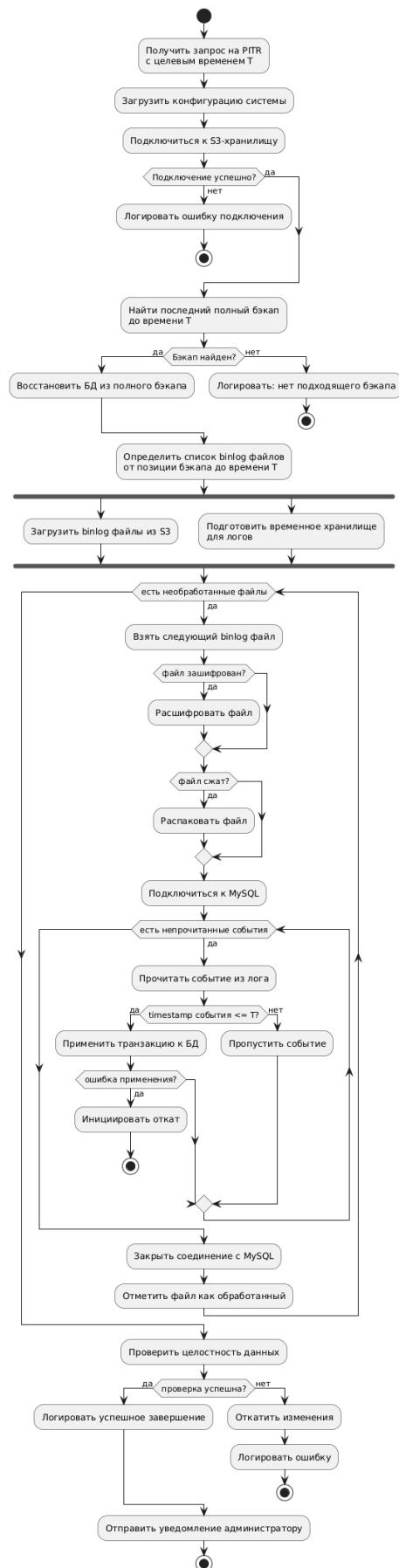


Рисунок 2 – Алгоритм восстановления на заданный момент времени

На втором шаге инструмент выбирает резервную копию, созданную не позже целевого момента. Если доступно несколько копий, выбирается наиболее поздняя копия, чтобы уменьшить количество бинарных журналов, которые потребуется применить. Такой принцип соответствует общей логике PITR: чем ближе исходная копия к целевой точке, тем меньше объем roll-forward и тем ниже RTO [1; 5].

Далее выполняется проверка цепочки бинарных журналов. Инструмент получает список binlog-файлов, определяет файл и позицию старта, затем проверяет наличие всех последующих файлов до целевой точки. Если обнаружен пропуск, восстановление останавливается до изменения состояния MySQL.

После проверки запускается восстановление полной резервной копии. При использовании WAL-G выполняется backup-fetch, а для xtrabackup-сценария дополнительно выполняется prepare-команда [7]. Затем MySQL стартует с параметрами, исключающими автоматический запуск репликации до завершения настройки.

Инициализация GTID выполняется до подключения к binlog-server. Если резервная копия содержит xtrabackup_binlog_info с GTID, инструмент устанавливает gtid_purged так, чтобы MySQL считал транзакции, входящие в резервную копию, уже выполненными. Документация MySQL описывает gtid_purged как множество GTID, которые были зафиксированы, но отсутствуют в бинарных журналах сервера [4].

Затем запускается binlog-server с параметром целевой точки. Восстановленный сервер настраивается командой CHANGE REPLICATION SOURCE TO или совместимой командой CHANGE MASTER TO в зависимости от версии MySQL. Завершение восстановления наступает, когда binlog-server достигает заданной точки и прекращает передачу событий, а MySQL применяет все полученные транзакции.

Для ограничения временного дискового использования алгоритм работает с бинарными журналами поэтапно. Перед применением очередного

участка восстановления инструмент получает только необходимый binlog-файл или минимальный набор файлов, передает события восстановленному MySQL через репликационный протокол, фиксирует прогресс и удаляет локальную копию после того, как она больше не требуется. Такой режим снижает риск переполнения диска при восстановлении через длинный диапазон журналов и делает возможной работу с отдельными binlog-файлами размером более 15 ГБ.

2.4. Реализация компонентов загрузки, хранения и отдачи бинарных журналов

Реализация инструмента состоит из нескольких модулей: модуль конфигурации, модуль работы с резервными копиями, модуль индекса бинарных журналов, модуль управления binlog-server, модуль управления MySQL и модуль состояния операции. Отдельная оболочка управления в рамках работы не разрабатывается: запуск выполняется через штатную команду `wal-g binlog-server` и параметры конфигурации WAL-G.

Модуль конфигурации читает параметры из файла, переменных окружения и аргументов командной строки. Для совместимости с WAL-G поддерживаются параметры подключения к MySQL, команды создания и восстановления backup, путь временного каталога binlog и настройки binlog-server [7]. Пример конфигурации представлен в приложении 1.

Модуль резервных копий отвечает за получение списка backup-объектов, выбор копии и запуск восстановления. В промышленном варианте выбор копии выполняется по метаданным: времени создания, времени завершения, типу копии, родительской полной копии для инкрементальных цепочек и доступности в хранилище.

Модуль индекса бинарных журналов строит каталог доступных binlog-файлов и хранит для каждого файла имя, размер, контрольную сумму, время архивации, диапазон GTID, начальную и конечную позицию, ссылку на объект и статус проверки. Если WAL-G используется в режиме `WALG_MYSQL_CHECK_GTIDS`, он может анализировать

PREVIOUS_GTIDS_EVENT для предотвращения повторной загрузки журналов с уже встречавшимися событиями [7].

Модуль управления binlog-server формирует команду запуска, проверяет занятость порта, анализирует логи, контролирует поэтапное получение бинарных журналов из хранилища и завершает процесс после достижения целевой точки. Временные файлы binlog удаляются после передачи и применения, что ограничивает потребление локального диска. Модуль управления MySQL выполняет подготовку конфигурации, запуск сервера, инициализацию GTID и настройку репликации. На практике этот модуль является самым чувствительным к настройкам: неверный server_id, пользователь без прав репликации или недоступный временный каталог приводят к остановке восстановления еще до применения журналов.

Ниже приведен пример конфигурации binlog-server, используемой при восстановлении:

1. WALG_MYSQL_BINLOG_SERVER_HOST=127.0.0.1
2. WALG_MYSQL_BINLOG_SERVER_PORT=9306
3. WALG_MYSQL_BINLOG_SERVER_USER=walg
4. WALG_MYSQL_BINLOG_SERVER_PASSWORD=<secret>
5. WALG_MYSQL_BINLOG_SERVER_ID=99
6. WALG_MYSQL_BINLOG_SERVER_REPLICA_SOURCE=root:<secret>@tcp(127.0.0.1:3306)/mysql
7. WALG_MYSQL_BINLOG_DST=/var/lib/mysql/walg-binlogs
8. WALG_S3_PREFIX=s3://example-bucket/mysql/

Для обработки ошибок реализованы типовые классы: StorageUnavailable, BackupNotFound, BinlogGapDetected, MySQLStartFailed, GTIDInitializationFailed, ReplicationError, TargetPointUnavailable и ValidationFailed. Каждая ошибка имеет пользовательское сообщение и технические детали.

При запуске восстановления должны проверяться ресурсы: свободное место, квоты на создание восстановленного экземпляра, доступность

хранилища, сетевые правила и параметры binlog-server. Ошибка нехватки ресурсов после частичного восстановления приводит к потере времени, поэтому предварительная проверка является обязательной частью restore plan. Хотя поэтапная загрузка снижает требования к диску, полностью игнорировать этот ресурс нельзя: место должно покрывать текущий binlog, распаковку, служебные файлы MySQL и запас на повторные попытки.

Для надежности необходимо различать ошибки, которые можно повторить автоматически, и ошибки, требующие вмешательства оператора. К первым относятся временная недоступность хранилища или сетевой таймаут. Ко вторым - разрыв цепочки binlog, несовместимость версии MySQL, повреждение резервной копии или ошибка репликации из-за конфликтующего GTID.

2.5. Порядок запуска и предварительная настройка wal-g binlog-server

В рамках данной работы не вводится дополнительная оболочка управления поверх WAL-G. Работа с разработанным компонентом выполняется через существующий способ запуска WAL-G: команду wal-g binlog-server и набор параметров конфигурации. Такой вариант проще для эксплуатации и лучше соответствует назначению компонента: он не управляет всей инфраструктурой восстановления, а отдает восстановленному MySQL бинарные журналы по протоколу репликации.

Перед запуском необходимо подготовить окружение. На восстановленном сервере MySQL должны быть включены GTID или корректно определены имя binlog-файла и позиция старта. Для пользователя репликации должны быть выданы права, достаточные для подключения к источнику репликации. Также заранее задаются адрес и порт binlog-server, server_id, путь временного каталога, параметры доступа к объектному хранилищу и строка подключения к MySQL. Без этой настройки binlog-server не сможет корректно принять подключение восстановленного экземпляра и определить, с какого места начинать передачу событий.

После запуска восстановленный MySQL настраивается как реплика источника, которым в этом сценарии выступает wal-g binlog-server. Сервер MySQL подключается к указанному адресу и получает события постепенно. Компонент читает нужные бинарные журналы из хранилища, передает их через репликационный протокол и освобождает временные файлы после использования. Благодаря этому не требуется предварительно скачивать всю цепочку binlog и не требуется собирать один большой SQL-файл. Пример настройки MySQL представлен в приложении 2.

Практически важным является контроль границы восстановления. Если восстановление выполняется на момент времени, binlog-server должен прекратить передачу событий после достижения целевой точки. Если используется позиция или GTID, граница определяется соответствующим набором метаданных. В любом случае до старта нужно проверить, что нужный диапазон бинарных журналов доступен в хранилище и не содержит разрывов.

Отдельно следует учитывать, что wal-g binlog-server не отменяет подготовительные действия. Он ускоряет применение журналов и снижает расход временного диска, но требует корректной настройки до восстановления: доступного объектного хранилища, валидных учетных данных, свободного порта, подходящего server_id, временного каталога и корректной конфигурации восстановленного MySQL. Поэтому в эксплуатационном сценарии запуск binlog-server должен сопровождаться предварительной проверкой параметров и анализом служебных логов.

2.6. Методика тестирования и анализ результатов

Тестирование инструмента должно подтвердить две группы свойств: корректность восстановления и эксплуатационную эффективность. Корректность означает, что восстановленная база соответствует выбранной точке: все изменения до нее присутствуют, изменения после нее отсутствуют, транзакционные границы не нарушены. Эксплуатационная эффективность в данной работе трактуется уже: инструмент должен ускорять этап применения

binlog и снижать максимум временного дискового пространства по сравнению со сценариями, где журналы заранее скачиваются или преобразуются в цельный SQL-поток.

Тестовая среда для проверки может включать один исходный сервер MySQL 8.0 или 8.4, объектное хранилище S3-совместимого типа, WAL-G с режимом binlog-server и восстановленный сервер MySQL. Для генерации нагрузки применяются sysbench или пользовательский набор транзакций. Перед запуском тестов включаются binary logging, GTID и row-based формат binlog [3; 4].

Таблица 3 – Методика тестирования инструмента восстановления

Сценарий	Цель	Проверяемый результат
Восстановление без нагрузки	Проверить базовую работоспособность	База стартует, ошибки репликации отсутствуют
Массовая запись	Проверить применение большого объема binlog	Транзакции до target_time присутствуют
Крупный binlog > 15 ГБ	Проверить работу без цельного SQL-файла	Восстановление проходит поэтапно, временный каталог не содержит всю цепочку
Ошибка DELETE после target_time	Проверить точность остановки	Удаление после точки восстановления отсутствует
Разрыв цепочки binlog	Проверить fail-fast	Операция останавливается до применения

Сценарий	Цель	Проверяемый результат
Перезапуск Recovery Manager	Проверить устойчивость	Операция возобновляется или помечается ошибкой
Сравнение mysqlbinlog и binlog-server	Оценить производительность	Измерены RTO, CPU, объем временных файлов

Для проверки точности восстановления создается контрольная таблица pitr_events со столбцами id, event_time, payload и checksum. Нагрузочный процесс записывает события с временными метками и контрольными значениями. Затем выполняется искусственная ошибка, например удаление части данных после заранее известного момента. Восстановление запускается на точку перед ошибкой.

Для проверки целостности используются сравнение количества строк и контрольных сумм по таблицам, проверка GTID-множества восстановленного сервера, прикладные запросы и проверка отсутствия ошибок в SHOW REPLICA STATUS. Такой набор проверок снижает риск ложного успеха, когда сервер запустился, но данные восстановлены неполно.

Таблица 4 – Контрольные показатели для сравнения подходов восстановления

Подход	Поток данных	Влияние на RTO	Особенности
mysqlbinlog	binlog -> SQL/BINLOG -> mysql client	Базовый уровень	Простая схема, много преобразований
WAL-G binlog-replay	binlog из хранилища -> replay command	Среднее	Автоматизирует fetch/replay

Подход	Поток данных	Влияние на RTO	Особенности
WAL-G binlog-server	binlog из хранилища -> replication protocol -> MySQL	Наиболее перспективное	Не требует цельного SQL-файла, работает поэтапно и снижает расход временного диска

Для разрабатываемого инструмента основным преимуществом является не сама автоматизация PITR, поскольку WAL-G уже предоставляет binlog-replay, а сокращение RTO на этапе применения журналов. MySQL применяет события как поток репликации, а инструмент управляет доставкой бинарных журналов из хранилища. Нет необходимости формировать цельный SQL-файл или заранее держать на диске всю цепочку binlog. Дополнительный эффект состоит в снижении требований к временному диску: журналы скачиваются непосредственно перед применением и удаляются после использования.

Для каждой серии тестов необходимо указывать версию MySQL, версию WAL-G, размер базы, объем binlog, выбранную точку восстановления, время восстановления полной копии, время старта MySQL, время инициализации GTID, время применения binlog, суммарный RTO, максимальную загрузку CPU, объем временных файлов и результат проверки.

При сравнении подходов необходимо отдельно фиксировать время восстановления полной копии и время применения бинарных журналов. Именно второй показатель показывает эффект от отказа от формирования цельного SQL-представления и от поэтапной загрузки binlog. Дополнительно измеряется максимальный объем временного каталога: для разработанного сценария он должен оставаться близким к размеру одного текущего binlog-

файла или небольшого буфера, а не к суммарному объему всех журналов в диапазоне восстановления.

2.7. Оценка ограничений, рисков и направлений развития

Несмотря на преимущества binlog-server, предложенное решение имеет ограничения. Первое ограничение связано с непрерывностью бинарных журналов. Если binlog был удален до загрузки в хранилище или загрузился с ошибкой, PITR до точек после разрыва становится невозможным. Поэтому инструмент восстановления должен сопровождаться регламентом регулярной выгрузки binlog и мониторингом отставания между текущим состоянием MySQL и последним сохраненным журналом.

Второе ограничение связано с целевой точкой восстановления по времени. Временная точка удобна пользователю, но внутренне MySQL применяет транзакции и события. Если необходимо остановиться строго перед конкретной транзакцией, восстановление по GTID или позиции может быть точнее. Поэтому процедура восстановления должна поддерживать несколько способов указания границы: время, позиция и GTID.

Третье ограничение связано с совместимостью версий и терминологии MySQL. В MySQL 8.0/8.4 постепенно используется терминология source/replica, однако во многих инструментах и примерах сохраняются master/slave-команды. Инструмент должен генерировать команды, совместимые с целевой версией, и не полагаться на устаревшие параметры без проверки.

Четвертое ограничение связано с безопасностью. binlog содержит все изменения, включая потенциально чувствительные данные. Поэтому бинарные журналы в хранилище должны быть зашифрованы, доступ к ним должен быть ограничен, а временные файлы должны удаляться после завершения операции. WAL-G поддерживает сценарии хранения с компрессией и шифрованием, но конкретная политика безопасности зависит от инфраструктуры [7; 8].

Перспективы развития инструмента включают поддержку автоматического выбора оптимальной копии по расчетной стоимости восстановления, предварительную проверку PITR-диапазона, визуализацию доступных точек восстановления, интеграцию с системой алертинга, поддержку восстановления отдельных баз или таблиц, а также автоматическое создание тестовых восстановлений для регулярной проверки резервных копий.

Еще одно направление развития - построение каталога binlog-метаданных. Если при загрузке каждого binlog извлекать диапазон GTID, временные границы, контрольные суммы и статистику событий, то восстановление можно планировать быстрее и безопаснее. Такой каталог позволит заранее определить, какие точки доступны, где есть разрывы и сколько данных потребуется прочитать из хранилища.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была рассмотрена задача разработки инструмента для восстановления базы данных MySQL на заданный момент времени. Актуальность задачи обусловлена тем, что современные информационные системы предъявляют высокие требования к сохранности данных, а восстановление на точку во времени является одной из ключевых функций надежной эксплуатации баз данных.

В первой главе были изучены теоретические основы резервного копирования и восстановления баз данных. Показано, что PITR в MySQL основан на восстановлении полной резервной копии и последующем применении бинарных журналов. Рассмотрены роли redo log, undo log, binary log, GTID и позиции бинарного журнала. Проведен сравнительный анализ mysqldump, MyDumper, Percona XtraBackup и WAL-G.

Во второй главе была предложена архитектура инструмента восстановления на основе WAL-G и компонента binlog-server. Разработанный подход включает управляющий слой Recovery Manager, механизм выбора резервной копии, проверку непрерывности бинарных журналов, запуск восстановленного MySQL, инициализацию GTID, запуск binlog-server и контроль применения событий до целевой точки.

Были сформулированы функциональные и нефункциональные требования к инструменту, включая поддержку восстановления по времени, позиции и GTID, безопасность учетных данных, наблюдаемость, идемпотентность и обработку ошибок. Описан порядок предварительной настройки и запуска wal-g binlog-server в составе WAL-G без введения дополнительной оболочки управления.

Методика тестирования включает проверку корректности восстановления, точности остановки на целевой точке, поведения при разрыве цепочки binlog, устойчивости к перезапуску управляющего процесса

и сравнения с классическим mysqlbinlog-подходом. На основании анализа документации и опубликованных практических данных можно сделать вывод, что binlog-server является перспективным методом ускорения и автоматизации PITR [7].

Поставленная цель работы достигнута: спроектирован и описан инструмент восстановления MySQL на заданный момент времени, в котором разработанный компонент binlog-server используется для отдачи бинарных журналов восстановленному экземпляру MySQL по репликационному протоколу. Практическая ценность результата заключается в возможности применения подхода в промышленной инфраструктуре эксплуатации MySQL, где требуется надежное хранение резервных копий и бинарных журналов, ускоренное применение binlog и контролируемый порядок запуска восстановления.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

1. БД — база данных
2. СУБД — система управления базами данных
3. ПО — программное обеспечение
4. НИР — научно-исследовательская работа
5. PITR (Point-in-Time Recovery) — восстановление базы данных на заданный момент времени
6. WAL (Write-Ahead Logging) — протокол упреждающей записи
7. binlog — бинарный журнал транзакций MySQL
8. API (Application Programming Interface) — программный интерфейс приложения
9. CLI (Command Line Interface) — интерфейс командной строки
10. JSON (JavaScript Object Notation) — формат обмена данными
11. CI/CD (Continuous Integration / Continuous Delivery) — непрерывная интеграция и доставка
12. CPU (Central Processing Unit) — центральный процессор
13. RAM (Random Access Memory) — оперативная память
14. I/O (Input/Output) — операции ввода-вывода

СЛОВАРЬ ТЕРМИНОВ

1. Бинарный журнал MySQL - последовательность файлов, в которые сервер записывает события, изменяющие данные, и которые используются для репликации и восстановления [6].
2. Восстановление на точку во времени - процедура восстановления базы данных до состояния, соответствующего выбранному моменту после создания полной резервной копии [4].
3. GTID-множество - набор глобальных идентификаторов транзакций, позволяющий определить, какие транзакции были выполнены или должны быть применены [7].
4. Идемпотентность операции восстановления - свойство, при котором повторный запуск управляющего процесса не приводит к повторному применению уже примененных транзакций.
5. Объектное хранилище - хранилище, в котором данные размещаются в виде объектов и доступны по ключам.
6. Полная резервная копия - копия данных, достаточная для запуска базы данных в состоянии на момент создания копии.
7. Репликационный протокол MySQL - механизм передачи событий от источника к реплике, используемый в binlog-server-сценарии.
8. Целевая точка восстановления - время, позиция бинарного журнала или GTID-граница, до которой необходимо восстановить состояние базы.

СПИСОК ЛИТЕРАТУРЫ

1. MySQL 8.4 Reference Manual. Point-in-Time Recovery Using Binary Log [Электронный ресурс]. - URL: <https://dev.mysql.com/doc/refman/8.4/en/point-in-time-recovery-binlog.html> (дата обращения: 03.02.2026).
2. MySQL 8.4 Reference Manual. mysqlbinlog - Utility for Processing Binary Log Files [Электронный ресурс]. - URL: <https://dev.mysql.com/doc/en/mysqlbinlog.html> (дата обращения: 01.03.2026).
3. MySQL 8.4 Reference Manual. Binary Logging Options and Variables [Электронный ресурс]. - URL: <https://dev.mysql.com/doc/refman/8.4/en/replication-options-binary-log.html> (дата обращения: 11.03.2026).
4. MySQL 8.4 Reference Manual. GTID Format and Storage [Электронный ресурс]. - URL: <https://dev.mysql.com/doc/refman/8.4/en/replication-gtids-concepts.html> (дата обращения: 12.03.2026).
5. Percona XtraBackup 8.4 Documentation. Point-in-time recovery [Электронный ресурс]. - URL: <https://docs.percona.com/percona-xtrabackup/8.4/point-in-time-recovery.html> (дата обращения: 25.03.2026).
6. Percona XtraBackup 8.4 Documentation. Work with binary logs [Электронный ресурс]. - URL: <https://docs.percona.com/percona-xtrabackup/8.4/working-with-binary-logs.html> (дата обращения: 04.04.2026).
7. WAL-G Documentation. WAL-G for MySQL [Электронный ресурс]. - URL: <https://wal-g.readthedocs.io/MySQL/> (дата обращения: 15.04.2026).
8. WAL-G: Archival and Restoration for databases in the Cloud [Электронный ресурс] // GitHub. - URL: <https://github.com/wal-g/wal-g> (дата обращения: 20.04.2026).

9. MyDumper Project. MyDumper documentation [Электронный ресурс]. - URL: <https://mydumper.github.io/mydumper/docs/html/> (дата обращения: 02.05.2026).
10. Silberschatz, A. Database System Concepts [Text] / A. Silberschatz, H. F. Korth, S. Sudarshan. - 7th ed. - New York: McGraw-Hill Education, 2019. - 1376 p.
11. Gray, J. Transaction Processing: Concepts and Techniques [Text] / J. Gray, A. Reuter. - San Francisco: Morgan Kaufmann, 1993. - 1070 p.

СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА

Рисунок 1– Use Case диаграмма восстановления MySQL на точку во времени с использованием WAL-G binlog-server.

Рисунок 2 – Алгоритм восстановления на заданный момент времени.

Таблица 1 – Сравнительный анализ средств резервного копирования и восстановления MySQL.

Таблица 2 – Функциональные и нефункциональные требования к инструменту.

Таблица 3 – Методика тестирования инструмента восстановления.

Таблица 4 – Контрольные показатели для сравнения подходов восстановления.

ПРИЛОЖЕНИЯ

Приложение 1. Пример файла конфигурации WAL-G для MySQL

```
WALG_MYSQL_BINLOG_SERVER_HOST=127.0.0.1
WALG_MYSQL_BINLOG_SERVER_PORT=9306
WALG_MYSQL_BINLOG_SERVER_USER=walg
WALG_MYSQL_BINLOG_SERVER_PASSWORD=<secret>
WALG_MYSQL_BINLOG_SERVER_ID=99
WALG_MYSQL_BINLOG_SERVER_REPLICA_SOURCE=root:<secret>@tcp(
127.0.0.1:3306)/mysql
WALG_MYSQL_BINLOG_DST=/var/lib/mysql/walg-binlogs
WALG_S3_PREFIX=s3://example-bucket/mysql/
```

Приложение 2. Пример SQL-команд настройки восстановленного MySQL как реплики binlog-server

```
SET GLOBAL SERVER_ID = 999;
CHANGE REPLICATION SOURCE TO
SOURCE_HOST = '127.0.0.1',
SOURCE_PORT = 9306,
SOURCE_USER = 'walg',
SOURCE_PASSWORD = '<secret>',
SOURCE_AUTO_POSITION = 1;
START REPLICATION;
SHOW REPLICATION STATUS\G
```